多重网格大作业报告

黄文翀 3200100006
数学与应用数学（强基计划）

浙江大学 数学科学学院

2023/5/23

1. 一维情形
2. 二维情形
3. 二维不规则情形
4. 程序设计
5. 代数多重网格

1 一维情形

2 二维情形

3 二维不规则情形

4 程序设计

5 代数多重网格

FMG 的重复迭代

理论上，一次完整的 FMG 就可以达到二阶精度，但是实践中，有进一步的提升空间。

假设 FMG 第一次迭代的结果为 $\mathbf{v}$，我们可以令 $\mathbf{f}' = \mathbf{f} - A\mathbf{v}$，然后将 $\mathbf{f}'$ 作为方程右边提供给求解器，重复 FMG 求得 $\mathbf{v}'$，此时 $\mathbf{v} + \mathbf{v}'$ 即为两次 FMG 的结果。如此反复，就可以让 FMG-Cycle 执行多次迭代，以达到更高的求解精度。

在一维情形中如果使用 full-weighting + 二次插值，那么一次 FMG 就足够达到机器精度了。二维情形则需要多次 FMG 以提高精度。

一维情形的二次插值

考虑用 $ih, (i+1)h, (i+2)h$ 处的点值插值得到 $ih + \frac{h}{2}$ 处的点值。事实上，三点唯一确定一个二次函数，直接待定系数法求出函数，再将 $ih + \frac{h}{2}$ 代入即可，公式为：

$$U_{2i+1}^{h/2} = \frac{3}{8}U_i^h + \frac{6}{8}U_{i+1}^h - \frac{1}{8}U_{i+2}^h + O(h^3). \tag{1}$$

一维情形的二次插值

考虑用 $ih, (i+1)h, (i+2)h$ 处的点值插值得到 $ih + \frac{h}{2}$ 处的点值。事实上，三点唯一确定一个二次函数，直接待定系数法求出函数，再将 $ih + \frac{h}{2}$ 代入即可，公式为：

$$U_{2i+1}^{h/2} = \frac{3}{8}U_i^h + \frac{6}{8}U_{i+1}^h - \frac{1}{8}U_{i+2}^h + O(h^3). \tag{1}$$

这样得到的公式关于 $x = ih + \frac{h}{2}$ 不对称，为了使插值系数更加对称，我们用 $(i-1)h, ih, (i+1)h$ 处的点值再做一次二次插值，得到：

$$U_{2i+1}^{h/2} = \frac{6}{8}U_i^h + \frac{3}{8}U_{i+1}^h - \frac{1}{8}U_{i-1}^h + O(h^3). \tag{2}$$

一维情形的二次插值

考虑用 $ih, (i+1)h, (i+2)h$ 处的点值插值得到 $ih + \frac{h}{2}$ 处的点值。事实上，三点唯一确定一个二次函数，直接待定系数法求出函数，再将 $ih + \frac{h}{2}$ 代入即可，公式为：

$$U_{2i+1}^{h/2} = \frac{3}{8}U_i^h + \frac{6}{8}U_{i+1}^h - \frac{1}{8}U_{i+2}^h + O(h^3). \tag{1}$$

这样得到的公式关于 $x = ih + \frac{h}{2}$ 不对称，为了使插值系数更加对称，我们用 $(i-1)h, ih, (i+1)h$ 处的点值再做一次二次插值，得到：

$$U_{2i+1}^{h/2} = \frac{6}{8}U_i^h + \frac{3}{8}U_{i+1}^h - \frac{1}{8}U_{i-1}^h + O(h^3). \tag{2}$$

将上面两式相加，再乘 $\frac{1}{2}$，即可得到中心对称的二次插值公式：

$$U_{2i+1}^{h/2} = \frac{9}{16}U_i^h + \frac{9}{16}U_{i+1}^h - \frac{1}{16}U_{i-1}^h - \frac{1}{16}U_{i+2}^h + O(h^3). \tag{3}$$

一维情形的二次插值

考虑用 $ih, (i+1)h, (i+2)h$ 处的点值插值得到 $ih + \frac{h}{2}$ 处的点值。事实上，三点唯一确定一个二次函数，直接待定系数法求出函数，再将 $ih + \frac{h}{2}$ 代入即可，公式为：

$$U_{2i+1}^{h/2} = \frac{3}{8}U_i^h + \frac{6}{8}U_{i+1}^h - \frac{1}{8}U_{i+2}^h + O(h^3). \tag{1}$$

这样得到的公式关于 $x = ih + \frac{h}{2}$ 不对称，为了使插值系数更加对称，我们用 $(i-1)h, ih, (i+1)h$ 处的点值再做一次二次插值，得到：

$$U_{2i+1}^{h/2} = \frac{6}{8}U_i^h + \frac{3}{8}U_{i+1}^h - \frac{1}{8}U_{i-1}^h + O(h^3). \tag{2}$$

将上面两式相加，再乘 $\frac{1}{2}$，即可得到中心对称的二次插值公式：

$$U_{2i+1}^{h/2} = \frac{9}{16}U_i^h + \frac{9}{16}U_{i+1}^h - \frac{1}{16}U_{i-1}^h - \frac{1}{16}U_{i+2}^h + O(h^3). \tag{3}$$

对于靠近边界的点，直接使用公式 (3.3) 会有访问越界风险，需要视点的位置使用不那么对称的 (3.1) 或 (3.2)。

一个理论问题

用 full-weighting + 二次插值效果很好，但在理论上会遇到一些问题——FMG 的二阶收敛性是如何证明的？

一个理论问题

用 full-weighting + 二次插值效果很好，但在理论上会遇到一些问题——FMG 的二阶收敛性是如何证明的？

Lemma 9.51！它依赖于 Lemma 9.50，而它又依赖于 variational properties：

$$I_{2h}^{h} = c(I_{h}^{2h})^{T}, \tag{4}$$

$$I_{h}^{2h} A^{h} I_{2h}^{h} = A^{2h}. \tag{5}$$

上面的性质只有当 I_h^{2h} 为 full-weighting 算子、I_{2h}^h 为线性插值算子时才成立！如何解决？

一个理论问题

用 full-weighting + 二次插值效果很好，但在理论上会遇到一些问题——FMG 的二阶收敛性是如何证明的？

Lemma 9.51！它依赖于 Lemma 9.50，而它又依赖于 variational properties：

$$I_{2h}^{h} = c(I_{h}^{2h})^{T}, \tag{4}$$

$$I_{h}^{2h} A^{h} I_{2h}^{h} = A^{2h}. \tag{5}$$

上面的性质只有当 I_h^{2h} 为 full-weighting 算子、I_{2h}^h 为线性插值算子时才成立！如何解决？

An open problem.

1 一维情形

2 二维情形

3 二维不规则情形

4 程序设计

5 代数多重网格

二维情形的二次插值

仍然将点分为四类。事实上，在粗网格线上的点可以使用一维的二次插值公式，唯一需要考虑的就是不在粗网格线上的点。考虑点 $P_0 = G^h(2i+1, 2j+1)$，做一次二维二次插值需要六个点，取：

$$P_1 = G^h(i,j) \qquad P_2 = G^h(i,j+1) \qquad P_3 = G^h(i,j+2)$$

$$P_4 = G^h(i+1,j) \qquad P_5 = G^h(i+1,j+1) \qquad P_6 = G^h(i+1,j+2)$$

二维情形的二次插值

仍然将点分为四类。事实上，在粗网格线上的点可以使用一维的二次插值公式，唯一需要考虑的就是不在粗网格线上的点。考虑点 $P_0 = G^h(2i+1, 2j+1)$，做一次二维二次插值需要六个点，取：

$$P_1 = G^h(i,j) \qquad P_2 = G^h(i,j+1) \qquad P_3 = G^h(i,j+2)$$

$$P_4 = G^h(i+1,j) \qquad P_5 = G^h(i+1,j+1) \qquad P_6 = G^h(i+1,j+2)$$

以这六个点的点值，通过待定系数法确定一个二维二次多项式 $Q(x,y) = A + Bx + Cy + Dx^2 + Exy + Fy^2$，再将 P_0 代入求得点值。结果为：

$$u(P_0) = \frac{1}{2}u(P_2) - \frac{1}{8}u(P_3) + \frac{1}{2}u(P_4) + \frac{1}{4}u(P_5) - \frac{1}{8}u(P_6).$$

二维情形的二次插值

仍然将点分为四类。事实上，在粗网格线上的点可以使用一维的二次插值公式，唯一需要考虑的就是不在粗网格线上的点。考虑点 $P_0 = G^h(2i+1, 2j+1)$，做一次二维二次插值需要六个点，取：

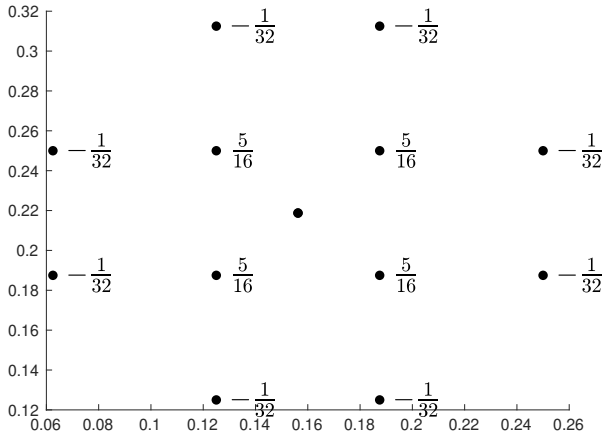
$$P_1 = G^h(i,j) \qquad P_2 = G^h(i,j+1) \qquad P_3 = G^h(i,j+2)$$

$$P_4 = G^h(i+1,j) \qquad P_5 = G^h(i+1,j+1) \qquad P_6 = G^h(i+1,j+2)$$

以这六个点的点值，通过待定系数法确定一个二维二次多项式 $Q(x,y) = A + Bx + Cy + Dx^2 + Exy + Fy^2$，再将 P_0 代入求得点值。结果为：

$$u(P_0) = \frac{1}{2}u(P_2) - \frac{1}{8}u(P_3) + \frac{1}{2}u(P_4) + \frac{1}{4}u(P_5) - \frac{1}{8}u(P_6).$$

这个形式不够对称，因此我们更换 $P_1, \ldots, P_6$ 的选取，重复上述过程三次，将每次得到的系数在各点上相加，并乘上 $\frac{1}{4}$，得到对称的二次插值公式，各点插值系数如下图所示。



与 LU 分解的求解速度对比

使用最佳组合 full-weighting + 二次插值，与 LU 分解对比求解耗时（秒），结果如下。

n	16	32	64	128	256	512	1024	增长阶数
V-Cycle	0.005	0.011	0.036	0.105	0.376	1.440	6.934	2.268
FMG-Cycle	0.002	0.015	0.025	0.090	0.351	1.182	3.835	1.698
LU 分解	0.002	0.022	0.42	8.168	-	-	-	4.281

理论分析——基本表述

与讲义一样，我们以齐次 Dirichlet 边值问题为例给出理论分析，此时的离散方程为 $A^h\mathbf{u}^h = \mathbf{f}^h$，系数矩阵 $A^h \in \mathbb{R}^{n^2\times n^2}$ 是分块矩阵，

$$A^h = \frac{1}{h^2}\begin{bmatrix} T & -I & & & \\ -I & T & -I & & \\ & -I & T & -I & \\ & & \ddots & \ddots & \ddots \\ & & & -I & T \end{bmatrix}, \qquad \text{其中} \quad T = \begin{bmatrix} 4 & -1 & & & \\ -1 & 4 & -1 & & \\ & -1 & 4 & -1 & \\ & & \ddots & \ddots & \ddots \\ & & & -1 & 4 \end{bmatrix} \in \mathbb{R}^{n\times n}.$$

根据讲义 Theorem 7.52，我们有：

定理

式 *(4.4)* 中矩阵 A^h 的特征值与对应的特征向量为：

$$\lambda_{ij} = \lambda_i + \lambda_j, \quad \mathbf{W}_{ij} = vec(\mathbf{w}_i\mathbf{w}_j^T), \quad i,j = 1,\dots,n-1. \tag{6}$$

其中

$$\lambda_k = \frac{4}{h^2}\sin^2\frac{kh\pi}{2}, \quad w_{k,j} = \sin(jkh\pi), \quad k,j = 1,\dots,n-1. \tag{7}$$

理论分析——基本表述

引理

式 *(4.4)* 中矩阵 A^h 的加权 *Jacobi* 迭代矩阵为：

$$T_\omega = (1-\omega)I + \omega D^{-1}(L+U) = I - \frac{\omega h^2}{4}A. \tag{8}$$

T_ω 的特征向量与 A^h 一样，对应的特征值为：

$$\lambda_{ij}(T_\omega) = 1 - \frac{\omega h^2}{4}\lambda_{ij} = 1 - \omega\sin^2\frac{ih\pi}{2} - \omega\sin^2\frac{jh\pi}{2}. \tag{9}$$

理论分析——基本表述

引理

式 *(4.4)* 中矩阵 A^h 的加权 *Jacobi* 迭代矩阵为:

$$T_\omega = (1-\omega)I + \omega D^{-1}(L+U) = I - \frac{\omega h^2}{4}A. \tag{8}$$

T_ω 的特征向量与 A^h 一样，对应的特征值为:

$$\lambda_{ij}(T_\omega) = 1 - \frac{\omega h^2}{4}\lambda_{ij} = 1 - \omega\sin^2\frac{ih\pi}{2} - \omega\sin^2\frac{jh\pi}{2}. \tag{9}$$

定义

二维 full-weighting 算子由下式给出。

$$v_{i,j}^{2h} = \frac{1}{16}(v_{2i-1,2j-1}^h + v_{2i-1,2j+1}^h + v_{2i+1,2j-1}^h + v_{2i+1,2j+1}^h) + \frac{1}{8}(v_{2i,2j-1}^h + v_{2i,2j+1}^h + v_{2i-1,2j}^h + v_{2i+1,2j}^h) + \frac{1}{4}v_{2i,2j}^h$$

其中 $i,j = 1,\ldots,n$. 用 $\mathbf{I}_h^{2h}$ 表示二维 full-weighting 算子。

理论分析——基本表述

引理

式 (4.4) 中矩阵 A^h 的加权 Jacobi 迭代矩阵为:

$$T_\omega = (1-\omega)I + \omega D^{-1}(L+U) = I - \frac{\omega h^2}{4}A. \tag{8}$$

T_ω 的特征向量与 A^h 一样，对应的特征值为:

$$\lambda_{ij}(T_\omega) = 1 - \frac{\omega h^2}{4}\lambda_{ij} = 1 - \omega\sin^2\frac{ih\pi}{2} - \omega\sin^2\frac{jh\pi}{2}. \tag{9}$$

定义

二维 full-weighting 算子由下式给出。

$$v_{i,j}^{2h} = \frac{1}{16}(v_{2i-1,2j-1}^h + v_{2i-1,2j+1}^h + v_{2i+1,2j-1}^h + v_{2i+1,2j+1}^h) + \frac{1}{8}(v_{2i,2j-1}^h + v_{2i,2j+1}^h + v_{2i-1,2j}^h + v_{2i+1,2j}^h) + \frac{1}{4}v_{2i,2j}^h$$

其中 $i,j = 1,\ldots,n$. 用 $\mathbf{I}_h^{2h}$ 表示二维 full-weighting 算子。

定义

二维线性插值算子 $\mathbf{I}_{2h}^h = 4(\mathbf{I}_h^{2h})^T$.

理论分析——TG 算子的谱半径

定义

对于 $i,j \in [1, \frac{n}{2})$，记 $i' = n - i, j' = n - j$，我们称 $\mathbf{W}_{ij}, \mathbf{W}_{ij'}, \mathbf{W}_{ij'}, \mathbf{W}_{i'j'}$ 为一组**互补波**。

理论分析——TG 算子的谱半径

定义

对于 $i,j \in [1, \frac{n}{2})$，记 $i' = n - i, j' = n - j$，我们称 $\mathbf{W}_{ij}, \mathbf{W}_{ij'}, \mathbf{W}_{ij'}, \mathbf{W}_{i'j'}$ 为一组**互补波**。

引理

二维 *full-weighting* 算子在一组互补波上作用的结果为：

$$\mathbf{I}_h^{2h}\mathbf{W}_{ij}^h = c_i c_j \mathbf{W}_{ij}^{2h},\quad \mathbf{I}_h^{2h}\mathbf{W}_{ij'}^h = -c_i s_j \mathbf{W}_{ij}^{2h},\quad \mathbf{I}_h^{2h}\mathbf{W}_{i'j}^h = -s_i c_j \mathbf{W}_{ij}^{2h},\quad \mathbf{I}_h^{2h}\mathbf{W}_{i'j'}^h = s_i s_j \mathbf{W}_{ij}^{2h}. \tag{10a}$$

其中 $i,j \in [1, \frac{n}{2}), i' = n - i, j' = n - j$. 另外

$$\mathbf{I}_h^{2h}\mathbf{W}_{i\frac{n}{2}}^h = \mathbf{0}, \quad i = 1, \ldots, n-1 \tag{10b}$$

$$\mathbf{I}_h^{2h}\mathbf{W}_{\frac{n}{2}j}^h = \mathbf{0}, \quad j = 1, \ldots, n-1. \tag{10c}$$

理论分析——TG 算子的谱半径

定义

对于 $i,j \in [1, \frac{n}{2})$，记 $i' = n-i, j' = n-j$，我们称 $\mathbf{W}_{ij}, \mathbf{W}_{ij'}, \mathbf{W}_{ij'}, \mathbf{W}_{i'j'}$ 为一组**互补波**。

引理

二维 *full-weighting* 算子在一组互补波上作用的结果为：

$$\mathbf{I}_h^{2h}\mathbf{W}_{ij}^h = c_i c_j \mathbf{W}_{ij}^{2h}, \quad \mathbf{I}_h^{2h}\mathbf{W}_{ij'}^h = -c_i s_j \mathbf{W}_{ij}^{2h}, \quad \mathbf{I}_h^{2h}\mathbf{W}_{i'j}^h = -s_i c_j \mathbf{W}_{ij}^{2h}, \quad \mathbf{I}_h^{2h}\mathbf{W}_{i'j'}^h = s_i s_j \mathbf{W}_{ij}^{2h}. \tag{10a}$$

其中 $i,j \in [1, \frac{n}{2}), i' = n-i, j' = n-j$. 另外

$$\mathbf{I}_h^{2h}\mathbf{W}_{i\frac{n}{2}}^h = \mathbf{0}, \quad i = 1, \ldots, n-1 \tag{10b}$$

$$\mathbf{I}_h^{2h}\mathbf{W}_{\frac{n}{2}j}^h = \mathbf{0}, \quad j = 1, \ldots, n-1. \tag{10c}$$

引理

线性插值算子在波上的作用结果为：

$$I_{2h}^h \mathbf{W}_{ij}^{2h} = c_i c_j \mathbf{W}_{ij}^h - c_i s_j \mathbf{W}_{ij'}^h - s_i c_j \mathbf{W}_{i'j}^h + s_i s_j \mathbf{W}_{i'j'}^h, \tag{11}$$

其中 $i,j \in [1, \frac{n}{2}), i' = n-i, j' = n-j$.

理论分析——TG 算子的谱半径

定理

$\mathcal{W}_{ij}^h = span\{\mathbf{W}_{ij}, \mathbf{W}_{ij'}, \mathbf{W}_{ij'}, \mathbf{W}_{i'j'}\}$ 是 *Two-Grid* 校正算子的不变子空间。

理论分析——TG 算子的谱半径

定理

$\mathcal{W}_{ij}^h = span\{\mathbf{W}_{ij}, \mathbf{W}_{ij'}, \mathbf{W}_{ij'}, \mathbf{W}_{i'j'}\}$ 是 *Two-Grid* 校正算子的不变子空间。

引理

取 $\nu_1 = \nu_2 = 3$，有 $\rho(TG) < 0.1$，因此能 *Two-Grid* 校正能够快速收敛。

理论分析——TG 算子的谱半径

定理

$\mathcal{W}_{ij}^h = span\{\mathbf{W}_{ij}, \mathbf{W}_{ij'}, \mathbf{W}_{ij'}, \mathbf{W}_{i'j'}\}$ 是 *Two-Grid* 校正算子的不变子空间。

引理

取 $\nu_1 = \nu_2 = 3$，有 $\rho(TG) < 0.1$，因此能 *Two-Grid* 校正能够快速收敛。

事实上，根据定理 4.2，我们有：

$$TG \begin{bmatrix} \mathbf{W}_{ij}^h \\ \mathbf{W}_{ij'}^h \\ \mathbf{W}_{i'j}^h \\ \mathbf{W}_{i'j'}^h \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \\ c_{41} & c_{42} & c_{43} & c_{44} \end{bmatrix} \begin{bmatrix} \mathbf{W}_{ij}^h \\ \mathbf{W}_{ij'}^h \\ \mathbf{W}_{i'j}^h \\ \mathbf{W}_{i'j'}^h \end{bmatrix} \tag{12}$$

只需对每个 $i, j \in [0, \frac{n}{2})$，求出相应的 $C = (c_{ij})$，计算 $\rho(C)$ 并取最大值即可得到 $\rho(TG)$。

理论分析——插值与限制算子的代数性质

要想证明 FMG 的收敛性，除了 TG 的谱半径，还需要 variational properties：

$$I_{2h}^{h} = c(I_{h}^{2h})^{T}, \tag{13}$$

$$I_{h}^{2h} A^{h} I_{2h}^{h} = A^{2h}. \tag{14}$$

第二条成立吗？

理论分析——插值与限制算子的代数性质

要想证明 FMG 的收敛性，除了 TG 的谱半径，还需要 variational properties：

$$I_{2h}^{h} = c(I_{h}^{2h})^{T}, \tag{13}$$

$$I_{h}^{2h} A^{h} I_{2h}^{h} = A^{2h}. \tag{14}$$

第二条成立吗？

不成立！

理论分析——插值与限制算子的代数性质

引理

在 Ω^h 中的网格点 (i,j) 上，*Laplace* 方程可以用九点离散表示：

$$\begin{aligned}\mathcal{L}u_{ij} =& \frac{1}{3h^2}(8u_{ij} - u_{(i-1)(j-1)} - u_{(i-1)j} - u_{(i-1)(j+1)} - u_{i(j-1)} - u_{i(j+1)} \\ & - u_{(i+1)(j-1)} - u_{(i+1)j} - u_{(i+1)(j+1)}).\end{aligned} \tag{15}$$

由泰勒展开容易得到：$\mathcal{L}u_{ij} = -\Delta u(x_i, y_j) + O(h^4)$.

我们设 B^h 为采用九点离散格式时的系数矩阵，有：

$$B^h = \frac{1}{h^2}\begin{bmatrix} K & P & & & \\ P & K & P & & \\ & P & K & P & \\ & & \ddots & \ddots & \ddots \\ & & & P & K \end{bmatrix}. \tag{16}$$

$$K = \begin{bmatrix} 8 & -1 & & & \\ -1 & 8 & -1 & & \\ & -1 & 8 & -1 & \\ & & \ddots & \ddots & \ddots \\ & & & -1 & 8 \end{bmatrix} \in \mathbb{R}^{n\times n}, \qquad P = \begin{bmatrix} -1 & -1 & & & \\ -1 & -1 & -1 & & \\ & -1 & -1 & -1 & \\ & & \ddots & \ddots & \ddots \\ & & & -1 & -1 \end{bmatrix} \in \mathbb{R}^{n\times n}$$

理论分析——插值与限制算子的代数性质

引理

线性插值算子与 *full-weighting* 算子关于 B^h 具有如下共轭性质：

$$\mathbf{I}_h^{2h} B^h \mathbf{I}_{2h}^h = B^{2h}. \tag{17}$$

理论分析——FMG 的收敛性

定义

对于一个正定对称矩阵 A，我们有“A 能量模”：$||\mathbf{u}||_A := \langle \mathbf{u}, \mathbf{u} \rangle_A = \langle A\mathbf{u}, \mathbf{u} \rangle$.

理论分析——FMG 的收敛性

定义

对于一个正定对称矩阵 A，我们有“A 能量模”：$||\mathbf{u}||_A := \langle \mathbf{u}, \mathbf{u} \rangle_A = \langle A\mathbf{u}, \mathbf{u} \rangle$.

引理

二维线性插值带来的误差在 B^h 能量模下，具有如下等式。

$$||\mathbf{I}_{2h}^h \mathbf{v}^{2h} - \mathbf{I}_{2h}^h \mathbf{u}^{2h}||_{B^h} = 2||\mathbf{v}^{2h} - \mathbf{u}^{2h}||_{B^{2h}}. \tag{18}$$

理论分析——FMG 的收敛性

定义

对于一个正定对称矩阵 A，我们有 “A 能量模”：$||\mathbf{u}||_A := \langle \mathbf{u}, \mathbf{u} \rangle_A = \langle A\mathbf{u}, \mathbf{u} \rangle$.

引理

二维线性插值带来的误差在 B^h 能量模下，具有如下等式。

$$||\mathbf{I}_{2h}^h \mathbf{v}^{2h} - \mathbf{I}_{2h}^h \mathbf{u}^{2h}||_{B^h} = 2||\mathbf{v}^{2h} - \mathbf{u}^{2h}||_{B^{2h}}. \tag{18}$$

引理

假设存在常数 $K > 0$，使得

$$||\mathbf{I}_{2h}^h \mathbf{u}^{2h} - \mathbf{u}^h||_{B^h} \leq Kh^2. \tag{19}$$

那么一次 *FMG-Cycle* 的代数误差有如下估计：

$$||\mathbf{e}^h||_{B^h} \leq Kh^2. \tag{20}$$

理论分析——FMG 的收敛性

定义

对于一个正定对称矩阵 A，我们有 "A 能量模"：$||\mathbf{u}||_A := \langle \mathbf{u}, \mathbf{u} \rangle_A = \langle A\mathbf{u}, \mathbf{u} \rangle$.

引理

二维线性插值带来的误差在 B^h 能量模下，具有如下等式。

$$||\mathbf{I}_{2h}^h \mathbf{v}^{2h} - \mathbf{I}_{2h}^h \mathbf{u}^{2h}||_{B^h} = 2||\mathbf{v}^{2h} - \mathbf{u}^{2h}||_{B^{2h}}. \tag{18}$$

引理

假设存在常数 $K > 0$，使得

$$||\mathbf{I}_{2h}^h \mathbf{u}^{2h} - \mathbf{u}^h||_{B^h} \leq Kh^2. \tag{19}$$

那么一次 *FMG-Cycle* 的代数误差有如下估计：

$$||\mathbf{e}^h||_{B^h} \leq Kh^2. \tag{20}$$

定理

对于二维齐次 *Dirichlet* 边值问题，只需要一次 *FMG-Cycle* 迭代，就能够达到 $O(h^2)$ 精度，计算耗时 $O(n \log n)$，n 是格点数。

1 一维情形

2 二维情形

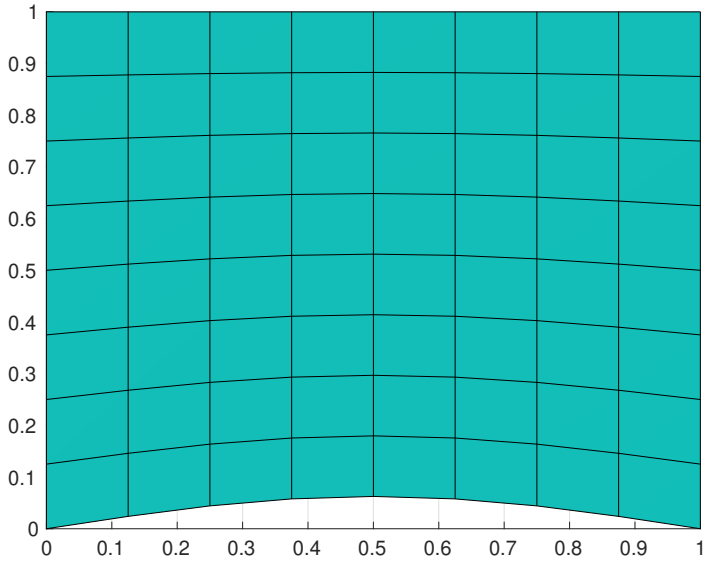
3 二维不规则情形

4 程序设计

5 代数多重网格

扭曲网格

将整个网格沿着不规则区域的形状扭曲，如下图所示。



在宽度为 $h = \frac{1}{n}$ 的扭曲网格中，网格点 $G^h(i,j)$ 的实际坐标为：

$$G^h(i,j) = (ih,\ D(ih) + jh(1 - D(ih))) \tag{21}$$

$$D(x) = \frac{1}{16}\sin(\pi x).$$

扭曲网格中的 Laplace 方程

在扭曲网格中，由于左右两个格点不在同一水平线上，因此二阶泰勒展开会存在 xy 交叉项。为将其消去，我们需要一个额外的的点。具体而言，记

$$P_0 = G^h(i,j) \qquad P_1 = G^h(i,j+1) \qquad P_2 = G^h(i,j-1)$$

$$P_3 = G^h(i+1,j) \qquad P_4 = G^h(i-1,j) \qquad P_5 = G^h(i+1,j+1)$$

扭曲网格中的 Laplace 方程

在扭曲网格中，由于左右两个格点不在同一水平线上，因此二阶泰勒展开会存在 xy 交叉项。为将其消去，我们需要一个额外的的点。具体而言，记

$$P_0 = G^h(i,j) \qquad P_1 = G^h(i,j+1) \qquad P_2 = G^h(i,j-1)$$
$$P_3 = G^h(i+1,j) \qquad P_4 = G^h(i-1,j) \qquad P_5 = G^h(i+1,j+1)$$

对于 $i = 0, ..., 5$，记 $P_i = (x_i, y_i)$，将 P_i 处的点值在 P_0 处做二阶 Taylor 展开：

$$u(P_i) = u(P_0) + (x_i - x_0)\frac{\partial u}{\partial x}(P_0) + (y_i - y_0)\frac{\partial u}{\partial y}(P_0) + \frac{1}{2}(x_i - x_0)^2\frac{\partial^2 u}{\partial x^2}(P_0)$$
$$+ \frac{1}{2}(y_i - y_0)^2\frac{\partial^2 u}{\partial y^2}(P_0) + (x_i - x_0)(y_i - y_0)\frac{\partial^2 u}{\partial x \partial y}(P_0) + O(h^3).$$

扭曲网格中的 Laplace 方程

在扭曲网格中，由于左右两个格点不在同一水平线上，因此二阶泰勒展开会存在 xy 交叉项。为将其消去，我们需要一个额外的的点。具体而言，记

$$P_0 = G^h(i,j) \qquad P_1 = G^h(i,j+1) \qquad P_2 = G^h(i,j-1)$$
$$P_3 = G^h(i+1,j) \qquad P_4 = G^h(i-1,j) \qquad P_5 = G^h(i+1,j+1)$$

对于 $i = 0, ..., 5$，记 $P_i = (x_i, y_i)$，将 P_i 处的点值在 P_0 处做二阶 Taylor 展开：

$$u(P_i) = u(P_0) + (x_i - x_0)\frac{\partial u}{\partial x}(P_0) + (y_i - y_0)\frac{\partial u}{\partial y}(P_0) + \frac{1}{2}(x_i - x_0)^2\frac{\partial^2 u}{\partial x^2}(P_0)$$
$$+ \frac{1}{2}(y_i - y_0)^2\frac{\partial^2 u}{\partial y^2}(P_0) + (x_i - x_0)(y_i - y_0)\frac{\partial^2 u}{\partial x \partial y}(P_0) + O(h^3).$$

列出六元一次方程组，解出系数 $A_0, ..., A_6$，使得

$$\sum_{i=0}^{6} A_i u(P_i) = -\Delta u(P_0) + O(h)$$

扭曲网格中的 Laplace 方程

在扭曲网格中，由于左右两个格点不在同一水平线上，因此二阶泰勒展开会存在 xy 交叉项。为将其消去，我们需要一个额外的的点。具体而言，记

$$P_0 = G^h(i,j) \qquad P_1 = G^h(i,j+1) \qquad P_2 = G^h(i,j-1)$$
$$P_3 = G^h(i+1,j) \qquad P_4 = G^h(i-1,j) \qquad P_5 = G^h(i+1,j+1)$$

对于 $i = 0, ..., 5$，记 $P_i = (x_i, y_i)$，将 P_i 处的点值在 P_0 处做二阶 Taylor 展开：

$$u(P_i) = u(P_0) + (x_i - x_0)\frac{\partial u}{\partial x}(P_0) + (y_i - y_0)\frac{\partial u}{\partial y}(P_0) + \frac{1}{2}(x_i - x_0)^2\frac{\partial^2 u}{\partial x^2}(P_0)$$
$$+ \frac{1}{2}(y_i - y_0)^2\frac{\partial^2 u}{\partial y^2}(P_0) + (x_i - x_0)(y_i - y_0)\frac{\partial^2 u}{\partial x \partial y}(P_0) + O(h^3).$$

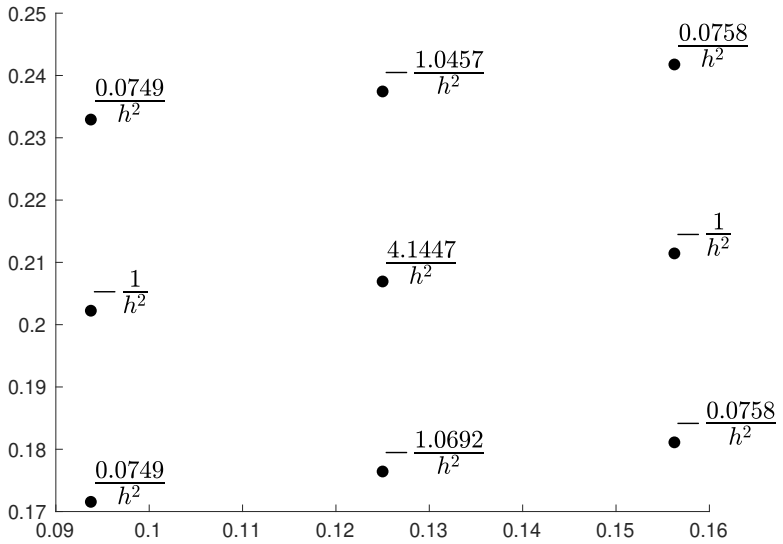
列出六元一次方程组，解出系数 $A_0, ..., A_6$，使得

$$\sum_{i=0}^{6} A_i u(P_i) = -\Delta u(P_0) + O(h)$$

这样列出的方程不够美观。因此我们更换 P_5 的选取，分别选择 $G^h(i+1,j-1), G^h(i-1,j+1), G^h(i-1,j-1)$ 重复上述过程，一共求得四组系数，将各组系数在对应点处相加，再乘以 $\frac{1}{4}$，即可得到更加对称的离散化 Laplace 方程。特别地，在靠近边界的格点处，上述过程会存在越界风险，因此需要视格点位置做特殊处理。

扭曲网格中的 Laplace 方程

以 $n = 32, i = 4, j = 6$ 为例，通过上述方法求得的各点系数如下图所示，可以看到大体上与规则网格类似。



扭曲网格中的 Neumann 边界条件

这里仅以 $x=0$ 边界为例，其它边界的想法时类似的。在扭曲网格中，$U_{0,j}, U_{1,j}, U_{2,j}$ 这三点之间存在 y 方向的偏移，因此二阶 Taylor 展式中存在 y, y^2, xy 项，这就需要额外的三个点来消除。与 Laplace 方程的方法完全类似，选取六个点：

$$P_0 = G^h(0,j) \qquad P_1 = G^h(1,j) \qquad P_2 = G^h(2,j)$$

$$P_3 = G^h(0,j-1) \qquad P_4 = G^h(0,j+1) \qquad P_5 = G^h(1,j+1).$$

扭曲网格中的 Neumann 边界条件

这里仅以 $x=0$ 边界为例，其它边界的想法时类似的。在扭曲网格中，$U_{0,j}, U_{1,j}, U_{2,j}$ 这三点之间存在 y 方向的偏移，因此二阶 Taylor 展式中存在 y, y^2, xy 项，这就需要额外的三个点来消除。与 Laplace 方程的方法完全类似，选取六个点：

$$P_0 = G^h(0,j) \qquad P_1 = G^h(1,j) \qquad P_2 = G^h(2,j)$$
$$P_3 = G^h(0,j-1) \qquad P_4 = G^h(0,j+1) \qquad P_5 = G^h(1,j+1).$$

对于 $i=0,\ldots,5$，我们记 $P_i=(x_i,y_i)$，将 P_i 处的点值在 P_0 处做二阶 Taylor 展开：

$$u(P_i) = u(P_0) + (x_i - x_0)\frac{\partial u}{\partial x}(P_0) + (y_i - y_0)\frac{\partial u}{\partial y}(P_0) + \frac{1}{2}(x_i - x_0)^2\frac{\partial^2 u}{\partial x^2}(P_0)$$
$$+ \frac{1}{2}(y_i - y_0)^2\frac{\partial^2 u}{\partial y^2}(P_0) + (x_i - x_0)(y_i - y_0)\frac{\partial^2 u}{\partial x \partial y}(P_0) + O(h^3).$$

扭曲网格中的 Neumann 边界条件

这里仅以 $x=0$ 边界为例，其它边界的想法时类似的。在扭曲网格中，$U_{0,j}, U_{1,j}, U_{2,j}$ 这三点之间存在 y 方向的偏移，因此二阶 Taylor 展式中存在 y, y^2, xy 项，这就需要额外的三个点来消除。与 Laplace 方程的方法完全类似，选取六个点：

$$P_0 = G^h(0,j) \qquad P_1 = G^h(1,j) \qquad P_2 = G^h(2,j)$$
$$P_3 = G^h(0,j-1) \qquad P_4 = G^h(0,j+1) \qquad P_5 = G^h(1,j+1).$$

对于 $i=0,\ldots,5$，我们记 $P_i=(x_i,y_i)$，将 P_i 处的点值在 P_0 处做二阶 Taylor 展开：

$$u(P_i) = u(P_0) + (x_i - x_0)\frac{\partial u}{\partial x}(P_0) + (y_i - y_0)\frac{\partial u}{\partial y}(P_0) + \frac{1}{2}(x_i - x_0)^2\frac{\partial^2 u}{\partial x^2}(P_0)$$
$$+ \frac{1}{2}(y_i - y_0)^2\frac{\partial^2 u}{\partial y^2}(P_0) + (x_i - x_0)(y_i - y_0)\frac{\partial^2 u}{\partial x \partial y}(P_0) + O(h^3).$$

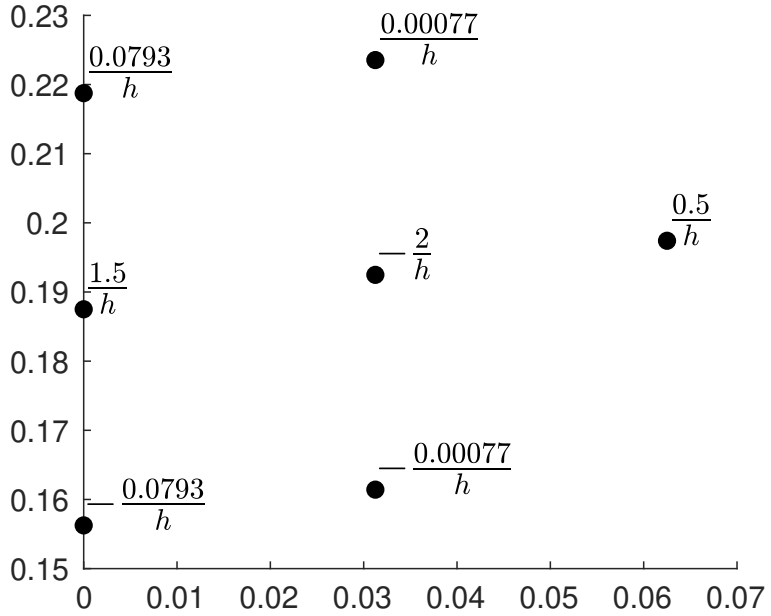
列出六元一次方程组，解出系数 $A_0,\ldots,A_6$，使得

$$\sum_{i=0}^{6} A_i u(P_i) = -u_x(P_0) + O(h^2)$$

为使方程更加对称，我们更换 P_5 的选取，取 $P_5 = G^h(1,j-1)$，重复上述过程，得到另一组系数，将各组系数在对应点处相加，再乘以 $\frac{1}{2}$，即可得到更加对称的 Neumann 边界条件方程。特别地，在靠近上下边界的格点处，上述过程会存在越界风险，因此需要视格点位置做特殊处理。

扭曲网格中的 Neumann 边界条件

以 $n = 32, j = 6$ 为例，通过上述方法求得的各点系数如下图所示，可以看到大体上与规则网格类似。



扭曲网格中的线性插值

对于一个细网格上的点，它的位置可能有四种情况：在粗网格点上、在粗网格的某条竖直线上、在粗网格的某条水平线上、不在粗网格线上。对于前两种情况，我们的处理方式与规则网格一样。对于第三种情况，因为扭曲网格的格点与左右两格点并不在同一直线上，因此需要引入第三点做线性插值。

扭曲网格中的线性插值

对于一个细网格上的点，它的位置可能有四种情况：在粗网格点上、在粗网格的某条竖直线上、在粗网格的某条水平线上、不在粗网格线上。对于前两种情况，我们的处理方式与规则网格一样。对于第三种情况，因为扭曲网格的格点与左右两格点并不在同一直线上，因此需要引入第三点做线性插值。

我们只介绍不在粗网格线上点如何插值。考虑点 $P_0 = G^h(2i+1, 2j+1)$，选取粗网格上三个点：

$$P_1 = G^{2h}(i,j) \qquad P_1 = G^{2h}(i,j+1) \qquad P_2 = G^{2h}(i+1,j)$$

扭曲网格中的线性插值

对于一个细网格上的点，它的位置可能有四种情况：在粗网格点上、在粗网格的某条竖直线上、在粗网格的某条水平线上、不在粗网格线上。对于前两种情况，我们的处理方式与规则网格一样。对于第三种情况，因为扭曲网格的格点与左右两格点并不在同一直线上，因此需要引入第三点做线性插值。

我们只介绍不在粗网格线上点如何插值。考虑点 $P_0 = G^h(2i+1, 2j+1)$，选取粗网格上三个点：

$$P_1 = G^{2h}(i,j) \qquad P_1 = G^{2h}(i,j+1) \qquad P_2 = G^{2h}(i+1,j)$$

对于 $i = 0, ..., 3$，我们记 $P_i = (x_i, y_i)$，将 P_i 处的点值在 P_0 处做一阶 Taylor 展开：

$$u(P_i) = u(P_0) + (x_i - x_0)\frac{\partial u}{\partial x}(P_0) + (y_i - y_0)\frac{\partial u}{\partial y}(P_0) + \frac{1}{2}(x_i - x_0)^2\frac{\partial^2 u}{\partial x^2}(P_0) + O(h^2).$$

扭曲网格中的线性插值

对于一个细网格上的点，它的位置可能有四种情况：在粗网格点上、在粗网格的某条竖直线上、在粗网格的某条水平线上、不在粗网格线上。对于前两种情况，我们的处理方式与规则网格一样。对于第三种情况，因为扭曲网格的格点与左右两格点并不在同一直线上，因此需要引入第三点做线性插值。

我们只介绍不在粗网格线上点如何插值。考虑点 $P_0 = G^h(2i+1, 2j+1)$，选取粗网格上三个点：

$$P_1 = G^{2h}(i,j) \qquad P_1 = G^{2h}(i,j+1) \qquad P_2 = G^{2h}(i+1,j)$$

对于 $i = 0, ..., 3$，我们记 $P_i = (x_i, y_i)$，将 P_i 处的点值在 P_0 处做一阶 Taylor 展开：

$$u(P_i) = u(P_0) + (x_i - x_0)\frac{\partial u}{\partial x}(P_0) + (y_i - y_0)\frac{\partial u}{\partial y}(P_0) + \frac{1}{2}(x_i - x_0)^2\frac{\partial^2 u}{\partial x^2}(P_0) + O(h^2).$$

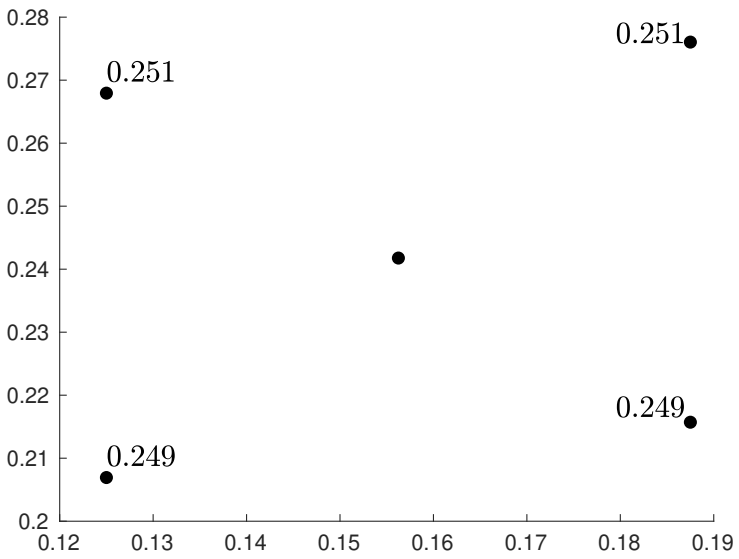
列出三元一次方程组，解出系数 $A_1, ..., A_3$，使得

$$\sum_{i=1}^{3} A_i u(P_i) = u(P_0) + O(h^2)$$

为了使插值系数更加对称，我们重新选取 P_1, P_2, P_3，每次在 P_0 周围的四个粗网格点上取三个点做上述过程，取遍所有可能，一共四次。将每次得到的系数在各点处相加并乘 $\frac{1}{4}$，即可得到更加对称的线性插值方程。

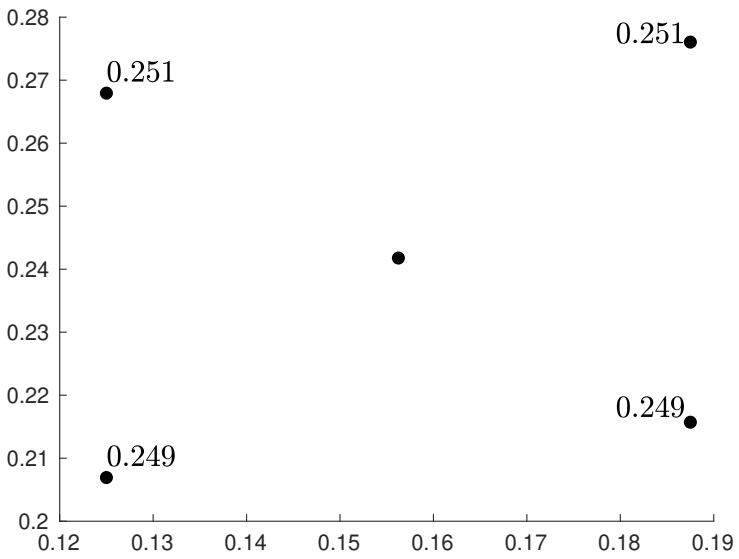
扭曲网格中的线性插值

以 $n = 32, i = 5, j = 7$ 为例，各点插值系数如下图所示，与规则网格的插值系数只相差大约 0.001.



扭曲网格中的线性插值

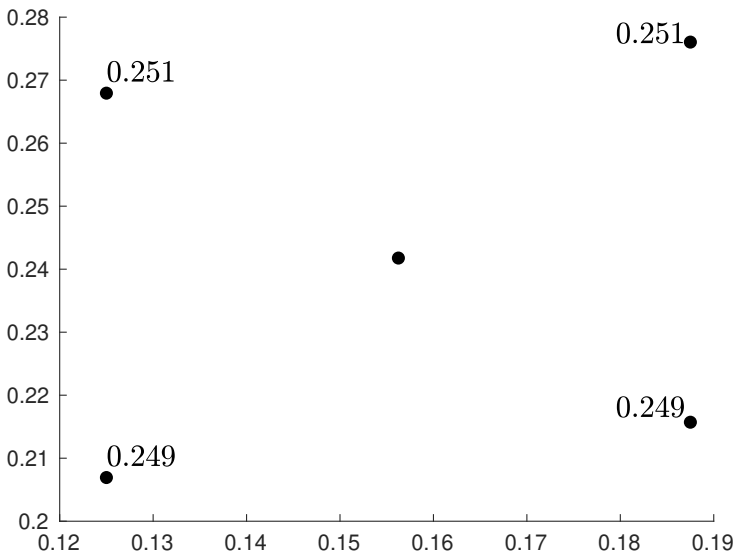
以 $n = 32, i = 5, j = 7$ 为例，各点插值系数如下图所示，与规则网格的插值系数只相差大约 0.001.



在最早先版本的代码中，我们实现了不规则网格的线性插值，当时我们并没有发现插值系数与规则网格如此相近。意识到这个情况后，在最终版本的代码中，出于计算性能考虑，我们选择直接使用规则网格的线性插值作为近似。

扭曲网格中的线性插值

以 $n = 32, i = 5, j = 7$ 为例，各点插值系数如下图所示，与规则网格的插值系数只相差大约 0.001.

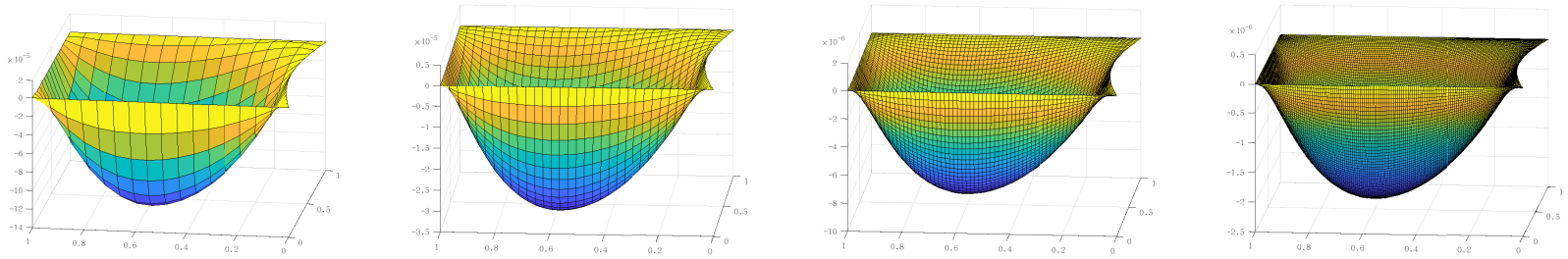


在最早先版本的代码中，我们实现了不规则网格的线性插值，当时我们并没有发现插值系数与规则网格如此相近。意识到这个情况后，在最终版本的代码中，出于计算性能考虑，我们选择直接使用规则网格的线性插值作为近似。

二次插值思路类似：Taylor 展开后求系数、对称化，不再赘述。

扭曲网格的效果

考虑由精确解 $u(x,y)=e^{\sin x+y}$ 导出的 Dirichlet 边值问题，用 $n=16,32,64,128,256,512,1024$ 的网格测试，使用 V-Cycle，限制算子选择 Full Operator，插值算子选择线性插值。当 $n=16,32,64,128$ 时，误差分布如下图所示。



可以看到，在 Dirichlet 条件下，格点越靠近边界，最终的计算误差越小。将数值解与真实解的误差用范数估计，并根据 $n=512$ 增加到 $n=1024$ 时误差减小的倍数，估计各范数下的收敛阶，结果如下表。

n	64	128	256	512	1024	收敛阶
$\|\|\cdot\|\|_1$	3.35×10^{-6}	8.72×10^{-7}	2.40×10^{-7}	5.42×10^{-8}	1.41×10^{-8}	1.944
$\|\|\cdot\|\|_2$	4.31×10^{-6}	1.11×10^{-6}	2.99×10^{-7}	6.86×10^{-8}	1.77×10^{-8}	1.952
$\|\|\cdot\|\|_\infty$	8.77×10^{-6}	2.23×10^{-6}	5.96×10^{-7}	1.38×10^{-7}	3.54×10^{-8}	1.958

大致呈二阶收敛。

1 一维情形

2 二维情形

3 二维不规则情形

4 程序设计

5 代数多重网格

参数管理

```
{
   "Condition Type" : "mixed",
   "Cycle Type" : "FMG",
   "Dimension" : 2,
   "Error Check" : true,
   "Grid Size" : 64,
   "Max Iter" : 10,
   "Prolongation" : "linear",
   "Reigeon Type" : "Irregular",
   "Restriction" : "injection",
   "eps" : 1e-08,
   "f" : "-(1-sin(x)+cos(x)*cos(x))*exp(sin(x)+y)",
   "g" : {
      "Down Bondary" : [ "cos(x)*exp(sin(x)+y)*(pi/16*cos(pi*x))/sqrt(pi*pi/256*(cos(pi*x))^2+1)+exp(sin(x)+y)*(-1)/sqrt(pi*pi/256*(cos(pi*x))^2+1)", "Neumann" ],
      "x=0" : [ "exp(sin(x)+y)", "Dirichlet" ],
      "x=1" : [ "exp(sin(x)+y)", "Dirichlet" ],
      "y=1" : [ "exp(sin(x)+y)", "Neumann" ]
   },
   "u" : "exp(sin(x)+y)"
}
```

各参数说明如下：

1. Condition Type 是条件的类型，您可以输入 Dirichlet，Neumann 或 mixed；
2. Dimension 是维数，您可以输入 1 或 2
3. Reigeon Type 是区域的类型，您可以输入 Regular 或 Irregular；
4. Grid Size 是网格的大小，您需要输入一个正整数，并且必须为 2 的正整数次幂；
5. f 用于描述 $-\Delta u$，g 用于描述边值条件；
6. Cycle Type：Cycle 的类型。您可以输入 V 或 FMG；
7. Restriction：限制算子的类型。您可以输入 injection 或 full operator；
8. Prolongation：插值算子的类型。您可以输入 linear 或 quadradic；
9. Max Iter：最大迭代次数。您可以输入一个正整数；
10. eps：求解精度需求。您可以输入一个正浮点数；
11. Check Error：若您需要验证误差，请设为 true，否则设为 false；
12. u：真实解。如果您需要验证误差，您必须提供真实解的表达式。

维数的抽象

一个根求解器 rootSolver，实现 V-Cycle、FMG-Cycle，使用的限制、插值算子是经过维数抽象后的对象，而 A^h 由子类在构造时生成。

两个子类 Solver1D 与 Solver2D 继承根求解器，子类只负责实现 A^h 的生成。

限制、插值算子内部根据 dim 判断是一维还是二维算子。

代码的大头是 A^h 的生成和各种插值算子，但是运行时间的大头是 Jacobi 迭代。90% 的时间花在不到 5% 的代码中。

表达式解析

为了方便用户能自定义方程而不需要重新编译，我们允许用户将方程以字符串的形式输入到 json 文件中，求解器通过表达式解析器读取。

```
Expression expr;                    // 定义解析器
expr.setExpr("exp(sin(x)+y)+C"); // 传入表达式
expr.addConstant("C", 2);           // 声明常量
expr.addVariable("x");              // 声明变量
expr.addVariable("y");
vector<double> parm(2);             // 以vector格式传入各变量值
parm[0] = parm[1] = 0.5;
cout << expr(parm) << endl;
```

上面是我们实现的表达式解析器的用法。其实用表达式解析器的效率远远低于把函数写在代码里编译，但因为 BVP 求解器中函数调用不是时间复杂度的主要部分，所以无所谓。非常不建议在 IVP 求解器中用这种手段。

稀疏矩阵

一个好的多重网格求解器应该具有 $O(n\log n)$ 的复杂度，其中 n 是格点的数量。如果使用稠密矩阵，那么 A^h 矩阵的存储本身就有 $\Theta(n^2)$ 的复杂度，是非常愚蠢的。

稀疏矩阵

一个好的多重网格求解器应该具有 $O(n\log n)$ 的复杂度，其中 n 是格点的数量。如果使用稠密矩阵，那么 A^h 矩阵的存储本身就有 $\Theta(n^2)$ 的复杂度，是非常愚蠢的。

可以使用 eigen 的稀疏矩阵（zzy 同学第一次大作业就用了）。或者自己实现。稀疏矩阵就是以 (i,j,c) 的方式存储矩阵元素，表示 $A_{ij}=c$，三元组的数量是 $O(n)$ 的。

稀疏矩阵

一个好的多重网格求解器应该具有 $O(n\log n)$ 的复杂度，其中 n 是格点的数量。如果使用稠密矩阵，那么 A^h 矩阵的存储本身就有 $\Theta(n^2)$ 的复杂度，是非常愚蠢的。

可以使用 eigen 的稀疏矩阵（zzy 同学第一次大作业就用了）。或者自己实现。稀疏矩阵就是以 (i,j,c) 的方式存储矩阵元素，表示 $A_{ij}=c$，三元组的数量是 $O(n)$ 的。

我们用了行压缩的存储方式，具体而言，我们将元素按行、列排好序，只存储 (j,c)，然后用一个 row_index 来记录每行的第一个元素下标。矩阵乘向量的只需做元素数量次乘法，是 $O(n)$ 的。

二次封装

考虑到求解器的设计比较复杂，对于用户并不友好，我们进行了二次封装。二次封装实现了一个类 BVP，这样主程序就非常简洁：

```
#include "BVP.h"
using namespace std;

int main(int argc, char* argv[]){
    BVP bvp;
    bvp.read(argv[1]);
    bvp.printProblem();
    bvp.solve();
    bvp.output("result.txt");
    vector<double>err = bvp.checkError();
    return 0;
}
```

1 一维情形

2 二维情形

3 二维不规则情形

4 程序设计

5 代数多重网格

背景与介绍

代数多重网格（AMG）是几何多重网格的一个自然推广，当我们发现由偏微分方程引出的离散方程组，使用多重网格具有如此优秀的表现时，自然会想，能不能将其推广到更一般的方程组，解更一般的大型稀疏矩阵。

背景与介绍

代数多重网格（AMG）是几何多重网格的一个自然推广，当我们发现由偏微分方程引出的离散方程组，使用多重网格具有如此优秀的表现时，自然会想，能不能将其推广到更一般的方程组，解更一般的大型稀疏矩阵。

答案是肯定的，这就是代数多重网格。具体而言，代数多重网格对 M 矩阵具有较好的表现，M 矩阵是满足正定对称、对角元为正数、非对角元非正的矩阵。事实上，大部分椭圆方程的离散矩阵都是 M 矩阵。

背景与介绍

代数多重网格（AMG）是几何多重网格的一个自然推广，当我们发现由偏微分方程引出的离散方程组，使用多重网格具有如此优秀的表现时，自然会想，能不能将其推广到更一般的方程组，解更一般的大型稀疏矩阵。

答案是肯定的，这就是代数多重网格。具体而言，代数多重网格对 M 矩阵具有较好的表现，M 矩阵是满足正定对称、对角元为正数、非对角元非正的矩阵。事实上，大部分椭圆方程的离散矩阵都是 M 矩阵。

对一个 $n \times n$ 的矩阵 A^h，我们不妨将 $1, \ldots, n$ 看作点，将 A^h 看作一个邻接矩阵，这样我们就得到了一个稀疏图，于是就有了网格结构。

背景与介绍

代数多重网格（AMG）是几何多重网格的一个自然推广，当我们发现由偏微分方程引出的离散方程组，使用多重网格具有如此优秀的表现时，自然会想，能不能将其推广到更一般的方程组，解更一般的大型稀疏矩阵。

答案是肯定的，这就是代数多重网格。具体而言，代数多重网格对 M 矩阵具有较好的表现，M 矩阵是满足正定对称、对角元为正数、非对角元非正的矩阵。事实上，大部分椭圆方程的离散矩阵都是 M 矩阵。

对一个 $n \times n$ 的矩阵 A^h，我们不妨将 $1, \ldots, n$ 看作点，将 A^h 看作一个邻接矩阵，这样我们就得到了一个稀疏图，于是就有了网格结构。

要想应用多重网格的思路，我们必须将方程限制到粗网格中，求解后返回细网格调整。但此时我们面临着严峻的问题：我们不知道网格的结构，只知道一个矩阵。我们需要一些算法来选取粗网格点 Ω^{2h}，并且定义限制算子 I_h^{2h} 与插值算子 I_{2h}^h，还要给出粗网格上的矩阵 A^{2h}。

选取粗网格点

要想从中选取粗网格点，首先给出如下定义：

若 i, j 满足：

$$|a_{i,j}| \geq \theta \max_{k \neq i} |a_{i,k}| \tag{22}$$

则称 i 强依赖于 j，也称 j 强影响 i. 其中 θ 是一个事先给定的阈值.

记 N_i 为所有使 $a_{i,j} \neq 0$ 的点 j 构成的集合，称为“相邻点”集。记 S_i 为所有强影响 i 的点构成的集合，S_i^T 为所有强依赖于 i 的点构成的集合。

在下文中，我们记粗网格 $\Omega^{2h} = C$，细网格 $\Omega^h = C \cup F$，即 F 表示所有在细网格但不在粗网格中的点。我们还记 $C_i = S_i \cap C$，$D_i^s = S_i \cap F$，$D_i^w = N_i \setminus S_i$.

选取粗网格点

我们希望用一种启发式的方法来选取粗网格点，具体地，我们希望：

1. $\forall i \in F$，对于 $j \in S_i$，或 $j \in C_i$，或 j 强依赖于 C_i 中某点；
2. C 中的点尽可能多，但需要保持：$\forall i \in C$，没有 $j \in C$ 使得 i 强依赖于 j。

选取粗网格点

我们希望用一种启发式的方法来选取粗网格点，具体地，我们希望：

1. $\forall i \in F$，对于 $j \in S_i$，或 $j \in C_i$，或 j 强依赖于 C_i 中某点；
2. C 中的点尽可能多，但需要保持：$\forall i \in C$，没有 $j \in C$ 使得 i 强依赖于 j。

当然，我们只是希望上述性质尽可能得到满足，为此，William L. Briggs 的书上给出了“染色算法”，如下：

1. $C, F \leftarrow \varnothing$, $\lambda_i \leftarrow |S_i^T|$.
2. 选取 $i \in \Omega^h \setminus (C \cup F)$，使 λ_i 最大.
3. $C \leftarrow C \cup \{i\}$, $F \leftarrow F \cup (S_i^T \setminus C)$.
4. $\forall j \in S_i^T$, $\forall k \in S_j \setminus (C \cup F)$, 令 $\lambda_k \leftarrow \lambda_k + 1$.
5. 若 $C \cup F = \Omega^h$，结束，否则返回第 2 步.

选取粗网格点

我们希望用一种启发式的方法来选取粗网格点，具体地，我们希望：

1. $\forall i \in F$，对于 $j \in S_i$，或 $j \in C_i$，或 j 强依赖于 C_i 中某点；
2. C 中的点尽可能多，但需要保持：$\forall i \in C$，没有 $j \in C$ 使得 i 强依赖于 j。

当然，我们只是希望上述性质尽可能得到满足，为此，William L. Briggs 的书上给出了“染色算法”，如下：

1. $C, F \leftarrow \varnothing$, $\lambda_i \leftarrow |S_i^T|$.
2. 选取 $i \in \Omega^h \setminus (C \cup F)$，使 λ_i 最大.
3. $C \leftarrow C \cup \{i\}$, $F \leftarrow F \cup (S_i^T \setminus C)$.
4. $\forall j \in S_i^T$, $\forall k \in S_j \setminus (C \cup F)$, 令 $\lambda_k \leftarrow \lambda_k + 1$.
5. 若 $C \cup F = \Omega^h$，结束，否则返回第 2 步.

注意到上述过程需要维护一个 λ_i 数组，支持增加、删除、求最大下标三种操作，我们期望单次操作的复杂度不超过 $O(\log n)$，因此我们使用二叉搜索树。当然，使用线段树、平衡树也是可以的，不过因为可以提前建树，所以平衡操作是不必要的。

构建插值算子

对于 $\mathbf{e} \in \Omega^{2h}$，插值算子具有如下格式：

$$(I_{2h}^{h}\mathbf{e})_i = \begin{cases} e_i & 若 i \in C, \\ \sum_{j \in C_i} \omega_{ij} e_j & 若 i \in F. \end{cases} \tag{23}$$

构建插值算子

对于 $\mathbf{e} \in \Omega^{2h}$，插值算子具有如下格式：

$$(I_{2h}^{h}\mathbf{e})_i = \begin{cases} e_i & 若 i \in C, \\ \sum_{j \in C_i} \omega_{ij} e_j & 若 i \in F. \end{cases} \tag{23}$$

在粗网格中，我们求解的方程是 $A^{2h}\mathbf{e} = \mathbf{f}$，其中 $\mathbf{f}$ 是光滑化（即若干次 G-S 迭代）后的残差，通常比较小，因此

$$a_{ii} e_i \approx -\sum_{j \in N_i} a_{ij} e_j .$$

构建插值算子

对于 $\mathbf{e} \in \Omega^{2h}$，插值算子具有如下格式：

$$(I_{2h}^{h}\mathbf{e})_i = \begin{cases} e_i & 若 i \in C, \\ \sum_{j \in C_i} \omega_{ij} e_j & 若 i \in F. \end{cases} \tag{23}$$

在粗网格中，我们求解的方程是 $A^{2h}\mathbf{e} = \mathbf{f}$，其中 $\mathbf{f}$ 是光滑化（即若干次 G-S 迭代）后的残差，通常比较小，因此

$$a_{ii} e_i \approx -\sum_{j \in N_i} a_{ij} e_j .$$

将 N_i 展开成三类，得

$$a_{ii} e_i \approx -\sum_{j \in C_i} a_{ij} e_j - \sum_{j \in D_i^s} a_{ij} e_j - \sum_{j \in D_i^w} a_{ij} e_j . \tag{24}$$

构建插值算子

对于 $\mathbf{e} \in \Omega^{2h}$，插值算子具有如下格式：

$$(I_{2h}^{h}\mathbf{e})_i = \begin{cases} e_i & 若 i \in C, \\ \sum_{j \in C_i} \omega_{ij} e_j & 若 i \in F. \end{cases} \tag{23}$$

在粗网格中，我们求解的方程是 $A^{2h}\mathbf{e} = \mathbf{f}$，其中 $\mathbf{f}$ 是光滑化（即若干次 G-S 迭代）后的残差，通常比较小，因此

$$a_{ii}e_i \approx -\sum_{j \in N_i} a_{ij}e_j.$$

将 N_i 展开成三类，得

$$a_{ii}e_i \approx -\sum_{j \in C_i} a_{ij}e_j - \sum_{j \in D_i^s} a_{ij}e_j - \sum_{j \in D_i^w} a_{ij}e_j. \tag{24}$$

对于 $j \in D_i^w$，将 e_j 近似为 e_i，得

$$\left(a_{ii} + \sum_{j \in D_i^w} a_{ij}\right) e_i \approx -\sum_{j \in C_i} a_{ij}e_j - \sum_{j \in D_i^s} a_{ij}e_j. \tag{25}$$

构建插值算子

对于 $\mathbf{e} \in \Omega^{2h}$，插值算子具有如下格式：

$$(I_{2h}^{h}\mathbf{e})_i = \begin{cases} e_i & 若 i \in C, \\ \sum_{j \in C_i} \omega_{ij} e_j & 若 i \in F. \end{cases} \tag{23}$$

在粗网格中，我们求解的方程是 $A^{2h}\mathbf{e} = \mathbf{f}$，其中 $\mathbf{f}$ 是光滑化（即若干次 G-S 迭代）后的残差，通常比较小，因此

$$a_{ii} e_i \approx -\sum_{j \in N_i} a_{ij} e_j .$$

将 N_i 展开成三类，得

$$a_{ii} e_i \approx -\sum_{j \in C_i} a_{ij} e_j - \sum_{j \in D_i^s} a_{ij} e_j - \sum_{j \in D_i^w} a_{ij} e_j . \tag{24}$$

对于 $j \in D_i^w$，将 e_j 近似为 e_i，得

$$\left(a_{ii} + \sum_{j \in D_i^w} a_{ij} \right) e_i \approx -\sum_{j \in C_i} a_{ij} e_j - \sum_{j \in D_i^s} a_{ij} e_j . \tag{25}$$

对于 $e_j \in D_i^s$，将其近似为

$$e_j \approx \frac{\sum_{k \in C_i} a_{jk} e_k}{\sum_{k \in C_i} a_{jk}} . \tag{26}$$

构建插值算子

对于 $\mathbf{e} \in \Omega^{2h}$，插值算子具有如下格式：

$$(I_{2h}^{h}\mathbf{e})_i = \begin{cases} e_i & 若 i \in C, \\ \sum_{j \in C_i} \omega_{ij} e_j & 若 i \in F. \end{cases} \tag{23}$$

在粗网格中，我们求解的方程是 $A^{2h}\mathbf{e} = \mathbf{f}$，其中 $\mathbf{f}$ 是光滑化（即若干次 G-S 迭代）后的残差，通常比较小，因此

$$a_{ii} e_i \approx -\sum_{j \in N_i} a_{ij} e_j .$$

将 N_i 展开成三类，得

$$a_{ii} e_i \approx -\sum_{j \in C_i} a_{ij} e_j - \sum_{j \in D_i^s} a_{ij} e_j - \sum_{j \in D_i^w} a_{ij} e_j . \tag{24}$$

对于 $j \in D_i^w$，将 e_j 近似为 e_i，得

$$\left(a_{ii} + \sum_{j \in D_i^w} a_{ij} \right) e_i \approx -\sum_{j \in C_i} a_{ij} e_j - \sum_{j \in D_i^s} a_{ij} e_j . \tag{25}$$

对于 $e_j \in D_i^s$，将其近似为

$$e_j \approx \frac{\sum_{k \in C_i} a_{jk} e_k}{\sum_{k \in C_i} a_{jk}} . \tag{26}$$

将 (6.5) 代入 (6.4)，得

$$\omega_{ij} = -\frac{a_{ij} + \sum_{m \in D_i^s} \left(\frac{a_{im} a_{mj}}{\sum_{k \in C_i} a_{mk}} \right)}{a_{ii} + \sum_{n \in D_i^w} a_{in}} \tag{27}$$

构建限制算子与粗网格矩阵 A^{2h}

事实上，有了插值算子，我们可以直接由对称性和 Garlerkin 条件构建 I_h^{2h} 与 A^{2h}，如下：

$$I_h^{2h} := (I_{2h}^h)^T \tag{28}$$

$$A^{2h} := I_h^{2h} A^h I_{2h}^h \tag{29}$$

这样 variational properties 总是自动满足。

构建 Cycle

代数多重网格的 Cycle 与几何多重是一样的，只不过使用的 A^h, I_h^{2h}, I_{2h}^h 都是事先构建好的。

谢谢

Thaks for Your Attention!